



WELCOME





C++ STL Sets

Agenda

C++ Bridge Course

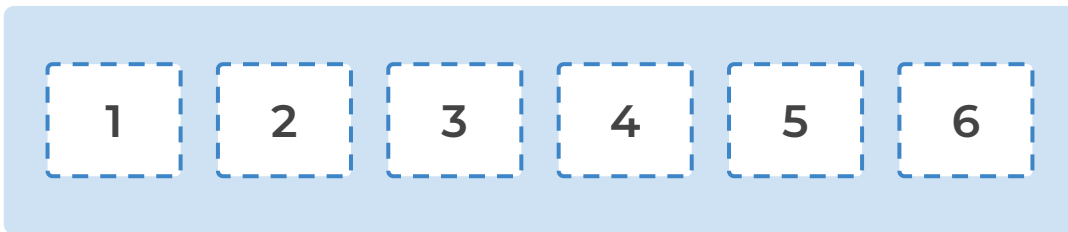
Set

Multi set

Unordered Set

Introduction

A **set** is a data structure that stores **unique** elements while maintaining **sorted order**.



Declaration and Initialization

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};
```

$$s = \{5, 10, 15, 20, 25, 30\}$$

*Initializing with **unique** values.*

Inserting Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
s.insert(35);
```

$$s = \{ 5, 10, 15, 20, 25, 30, 35 \}$$

Insert element (35) in the set.

Inserting Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
s.insert(35);  
s.insert(10);
```

$$s = \{5, 10, 15, 20, 25, 30, 35\}$$

Duplicate will NOT be added twice.

Inserting Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
s.insert(35);  
s.insert(10);  
s.insert(22);
```

$$s = \{ 5, 10, 15, 20, 22, 25, 30, 35 \}$$

s.insert(x) - inserts element *x* to *set*

Checking if an Element Exists

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
.  
if (s.find(25) != s.end()) {  
    cout << "25 is present";  
} else {  
    cout << "25 is not found";  
}
```

$$s = \{ 5, 10, 15, 20, 22, 25, 30, 35 \}$$

***s.find(x)** returns a pointer to element **x** in **set** if **present**.*

Else, return pointer to s.end()

Checking if an Element Exists

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
.  
if (s.find(25) != s.end()) {  
    cout << "25 is present";  
} else {  
    cout << "25 is not found";  
}
```

Output 

25 is present

Removing Element

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
s.erase(15);
```

$$s = \{ 5, 10, 15, 20, 22, 25, 30, 35 \}$$

$$s = \{ 5, 10, 20, 22, 25, 30, 35 \}$$

s.erase(x) - erases the element *x* if present in set

Removing Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
auto it1 = s.find(20);
```

$s = \{ 5, 10, 20, 22, 25, 30, 35 \}$

erase(it1, it2) removes all elements from *it1* to *it2*, excluding *it2* itself.

Removing Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};
```

```
·  
·
```

```
auto it1 = s.find(20);
```

```
auto it2 = s.find(35);
```

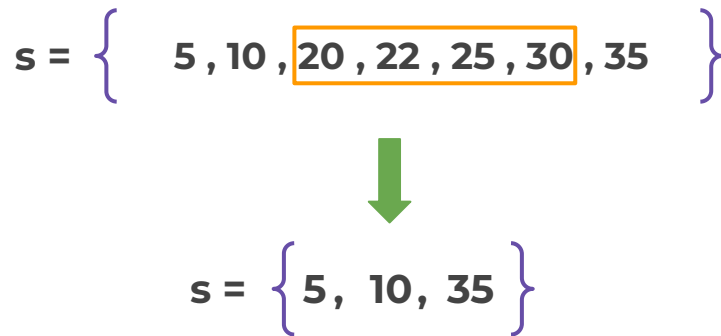
$s = \{ 5, 10, 20, 22, 25, 30, 35 \}$

erase(it1, it2) removes all elements from *it1* to *it2*, excluding *it2* itself.

Removing Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
auto it1 = s.find(20);  
auto it2 = s.find(35);  
s.erase(it1, it2);
```



erase(it1, it2) removes all elements from *it1* to *it2*, excluding *it2* itself.

Size of Set

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
auto it1 = s.find(20);  
auto it2 = s.find(35);  
s.erase(it1, it2);  
cout << s.size();
```

$$s = \{5, 10, 35\}$$
Output 

3

s.size() returns count of elements in set

Set

Printing All Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};
```

```
·  
·  
·
```

```
for (auto it = s.begin(); it != s.end(); it++) {  
    cout << *it << " ";  
}
```

$s = \{ 5, 10, 35 \}$

Output 

5 10 35

Removing All Elements

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};
```

```
·  
·  
·
```

```
s.clear();
```

$$s = \{5, 10, 35\}$$

$$s = \{ \}$$

s.clear() removes *all elements* from *set*

Checking if the Set is Empty

C++ 

```
set<int> s = {5, 10, 15, 20, 25, 30};  
.  
.  
.  
if (s.empty()) {  
    cout << "Set is empty";  
}
```

s = { }

Output 

Set is empty

s.empty(x) - returns **1** if set is empty. Else, returns **0**

Set

Swapping Two Sets

C++ 

```
set<int> s1 = {1, 2, 3};  
set<int> s2 = {4, 5, 6};
```

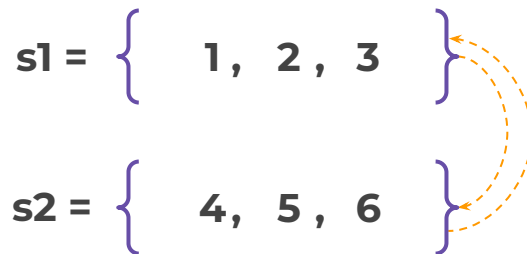
s1 = { 1, 2, 3 }

s2 = { 4, 5, 6 }

Swapping Two Sets

C++ 

```
set<int> s1 = {1, 2, 3};  
set<int> s2 = {4, 5, 6};  
s1.swap(s2);
```



Swapping Two Sets

C++ 

```
set<int> s1 = {1, 2, 3};  
set<int> s2 = {4, 5, 6};  
s1.swap(s2);
```

$s1 = \{ 4, 5, 6 \}$
 $s2 = \{ 1, 2, 3 \}$

Problem Statement

- ✓ Implement these functionality on a set -

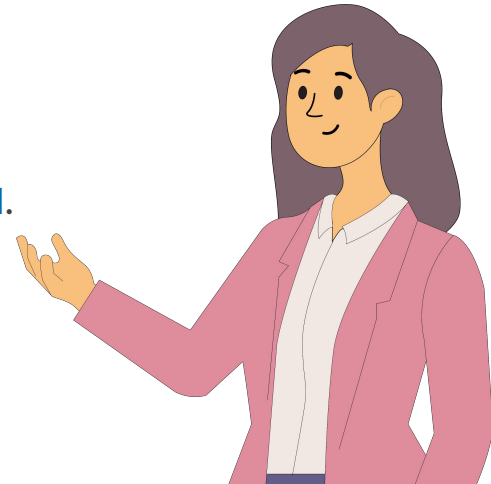
add x : Inserts element **x** into **set s**.

print : Prints all elements of **set s**, space-separated.

remove x : Removes element **x** from **set s**.

check x : Prints **1** if **x** is present in **s**, otherwise prints **-1**.

length : Prints the size of **set s**.



Problem Statement

Input



Queries = add 1
add 2
print
remove 2
check 3
check 1
length

Output



1 2

-1
1
1

Explanation

- **add 1** → **Inserts 1** into the set → Set: {1}
- **add 2** → **Inserts 2** into the set → Set: {1, 2}
- **print** → Prints the set contents: **1 2**
- **remove 2** → **Removes 2** from the set → Set: {1}
- **check 3** → **Checks if 3 is present** in the set → Output: -1 (since 3 is not in the set)
- **check 1** → **Checks if 1 is present** in the set → Output: 1 (since 1 is in the set)
- **length** → **Prints the size** of the set → Output: 1 (since set contain only 1 element)

Set Operations

Code

C++ 

```
#include <bits/stdc++.h>
using namespace std;
class SetOperations {
public:
    set<int> s;
    void insert(int x) {
        s.insert(x);
    }
    void print_contents() {
        for (int element : s) {
            cout << element << " ";
        }
        cout << endl;
    }
    ...
}
```

Set Operations

Code

C++ 

```
void erase(int x) {
    s.erase(x);
}

void find(int x) {
    if (s.find(x) != s.end()) {
        cout << 1 << endl;
    } else {
        cout << -1 << endl;
    }
}

void size() {
    cout << s.size() << endl;
}

};
```

Quiz Time!

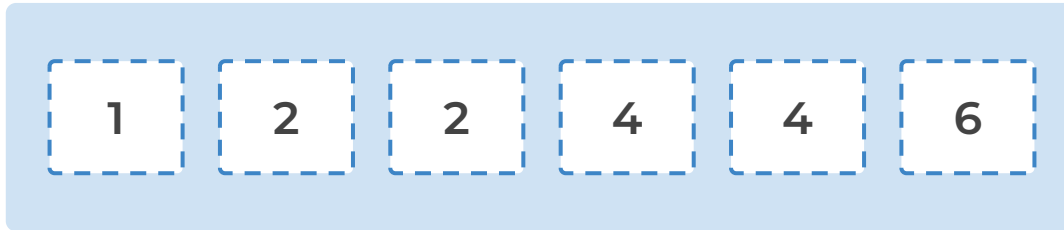




Multi set

Introduction

A **Multiset** is a modified version of set that allows **duplicate elements** while maintaining **sorted order**.



Declaration and Initialization

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};
```

$ms = \{ 5, 10, 10, 15, 20 \}$

Stored in *sorted order* (With *Duplicates Allowed*).

Multi set

Inserting Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
ms.insert(25);
```

$ms = \{ 5, 10, 10, 15, 20, 25 \}$

Insert element (25) in the multi set.

Multi set

Inserting Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
ms.insert(25);  
ms.insert(10);
```

$ms = \{ 5, 10, 10, 10, 15, 20, 25 \}$

Insert element (10) in the multi set (duplicates allowed).

Multi set

Inserting Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
ms.insert(25);  
ms.insert(10);  
ms.insert(30);
```

$ms = \{ 5, 10, 10, 10, 15, 20, 25, 30 \}$

Insert element (30) in the multi set.

Checking if an Element Exists

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
if (ms.find(10) != ms.end()) {  
    cout << "10 is present";  
} else {  
    cout << "10 is not found";  
}
```

$ms = \{ 5, 10, 10, 10, 15, 20, 25, 30 \}$

*`ms.find(x)` returns an iterator pointing to the **first occurrence** of `x` in the **multi set**. If `x` is not found, it returns an iterator to `ms.end()`.*

Multi set

Checking if an Element Exists

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
if (ms.find(10) != ms.end()) {  
    cout << "10 is present";  
} else {  
    cout << "10 is not found";  
}
```

Output 

10 is present

Count function of multi set

- ✓ The **count() function** returns the number of occurrences of a particular element in the multiset.
- ✓ It checks how many times the given element is present in the multiset.

Syntax:

```
ms.count(element);
```

Multi set

Count Function

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    multiset<int> ms = {10, 20, 10, 30, 20, 10};
    cout << ms.count(10);
}
```

Explanation:

- The **element 10** appears **3 times** in the multiset.
- So, **ms.count(10)** returns 3.

Output 

3

Removing Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
auto it = ms.find(10);
```

$ms = \{ 5, 10, 10, 10, 15, 20, 25, 30 \}$

Finds the *first occurrence* of 10.

Removing Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
auto it = ms.find(10);  
if (it != ms.end()) {  
    ms.erase(it);  
}
```

$ms = \{5, 10, 10, 10, 15, 20, 25, 30\}$



$ms = \{5, 10, 10, 15, 20, 25, 30\}$

`ms.erase(it)` - deletes the element pointed by iterator `it`.

Removing Elements

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
ms.erase(10);
```

$ms = \{5, 10, 10, 15, 20, 25, 30\}$



$ms = \{5, 15, 20, 25, 30\}$

`ms.erase(value)` - deletes all elements equal to value

Getting the Size of the Multi set

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
:  
cout << ms.size();
```

$ms = \{ 5, 15, 20, 25, 30 \}$

Output 

5

ms.size() returns the *number of elements* in the *multi set*.

Clearing the Multi set

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
ms.clear();
```

ms = { 5, 15, 20, 25, 30 }



ms = { }

Removes *all elements*.

Checking if the Multi set is Empty

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
:  
if (ms.empty()) {  
    cout << "Multiset is empty";  
}
```

$ms = \{ 5, 15, 20, 25, 30 \}$

Output 

Multiset is empty

ms.empty() returns 1 if multi set is empty. Else, return 0.

Swapping Two Multi sets

C++ 

```
multiset<int> ms1 = {1, 2, 3, 3};  
multiset<int> ms2 = {4, 5, 6};
```

ms1 = { 1, 2, 3, 3 }

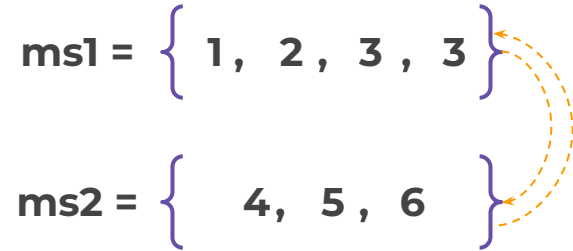
ms2 = { 4, 5, 6 }

Multi set

Swapping Two Multi sets

C++ 

```
multiset<int> ms1 = {1, 2, 3, 3};  
multiset<int> ms2 = {4, 5, 6};  
ms1.swap(ms2);
```



Multi set

Swapping Two Multi sets

C++ 

```
multiset<int> ms1 = {1, 2, 3, 3};  
multiset<int> ms2 = {4, 5, 6};  
ms1.swap(ms2);
```

ms1 = { 4, 5, 6 }

ms2 = { 1, 2, 3, 3 }

Problem Statement

You're given a **list of operations and a multiset**. Your task is to **perform** the following operations on **multi set**:

- **Insertion**: Add elements to the multi set.
- **Erase**: Remove elements from the multi set.
- **Find**: Locate an element in the multi set and print "**Found**" if present, "**Not Found**" otherwise.
- **Count**: Prints the occurrences of a specific element.
- **Erase First Occurrence**: Remove first instance of an element from multiset.

Problem Statement

Input



Multiset 1: [1, 2, 2, 3, 4]

Operations:

- insert 5
- erase 2
- find 3
- count 2
- eraseFirstOccurrence 3

Output



After Insertion: [1, 2, 2, 3, 4, 5]

After Erasing 2: [1, 3, 4, 5]

Find 3: Found

Count of 2: 0

After Erasing the first Occurrence of 3: [1, 4, 5]

Problem Statement

Insertion: Add 5 to the multi set while maintaining sorted order.

[1, 2, 2, 3, 4, 5]

Erase: Remove all occurrences of 2 from the multi set.

[1, 3, 4, 5]

Find: Search for 3 in the multi set and confirms its presence.

"Found"

Count: Determine the number of times 2 appears in the multi set, returning 0.

"Count of 2: 0"

Erase First Occurrence: Remove only the first occurrence of 3 from the multi set.

[1, 4, 5]

C++



```
#include <bits/stdc++.h>
using namespace std;
class MultiSetOperations {
public:
    multiset<int> ms;
    void insert(int value) {
        ms.insert(value);
    }
    void erase(int value) {
        ms.erase(value);
    }
}
```

C++



```
void findElement(int value) {
    auto it = ms.find(value);
    if (it != ms.end()) cout << "Found\n";
    else cout << "Not Found\n";
}

void countOccurrences(int value) {
    cout << "Count " << ms.count(value) << endl;
}

void eraseFirstOccurrence(int value) {
    auto it = ms.find(value);
    if(it != ms.end()) ms.erase(it);
}

};
```

Quiz Time!

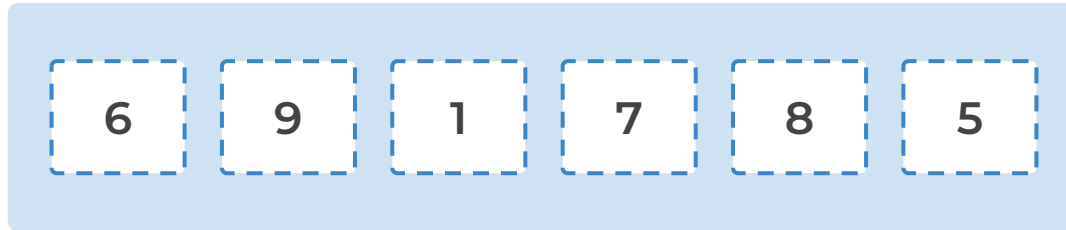




Unordered Set

Introduction

An **unordered_set** is a data structure that **stores unique elements** in **no particular order**.



Declaration and Initialization

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};
```

$us = \{10, 5, 25, 30, 15, 20\}$

Initializing with *unique values* (order is arbitrary).

Inserting Elements

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
us.insert(35);
```

$us = \{10, 5, 25, 30, 15, 20, 35\}$

Insert element (35) in the unordered_set.

Inserting Elements

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
us.insert(35);  
us.insert(10);
```

us = { 10, 5, 25, 30, 15, 20, 35 }

Duplicate will NOT be inserted.

Inserting Elements

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
us.insert(35);  
us.insert(10);  
us.insert(40);
```

us = {10, 5, 25, 30, 15, 20, 35, **40**}

Insert element (40) in the unordered_set.

Checking if an Element Exists

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
if (us.find(25) != us.end()) {  
    cout << "25 is present";  
} else {  
    cout << "25 is not found";  
}
```

us = {10, 5, 25, 30, 15, 20, 35, 40}

us.find(x) returns an iterator pointing to *x* in the *unordered_set*.

If *x* is not found, it returns an iterator to *us.end()*.

Checking if an Element Exists

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
:  
if (us.find(25) != us.end()) {  
    cout << "25 is present";  
} else {  
    cout << "25 is not found";  
}
```

Output 

25 is present

Removing One Element

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
us.erase(15);
```

$us = \{10, 5, 25, 30, 15, 20, 35, 40\}$



$us = \{10, 5, 25, 30, 20, 35, 40\}$

`us.erase(x)` deletes `x` from the `unordered_set`, if present.

Size of the Unordered Set

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
cout << us.size();
```

us = {10, 5, 25, 30, 20, 35, 40}

Output 

7

us.size() returns the *number of elements* in the *unordered_set*.

Clearing the Unordered Set

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
us.clear();
```

us = {10, 5, 25, 30, 20, 35, 40}



us = { }

us.clear() deletes all elements from the *unordered_set*

Checking if the Unordered Set is Empty

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
:  
if (us.empty()) {  
    cout << "Unordered set is empty";  
}
```

us = { }

us.empty() returns 1 if the *unordered_set* is *empty*; otherwise, *returns 0*.

Checking if the Unordered Set is Empty

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
if (us.empty()) {  
    cout << "Unordered set is empty";  
}
```

us = { }

Output 

Unordered set is empty

Swapping Two Unordered Sets

C++ 

```
unordered_set<int> us1 = {1, 2, 3};  
unordered_set<int> us2 = {4, 5, 6};
```

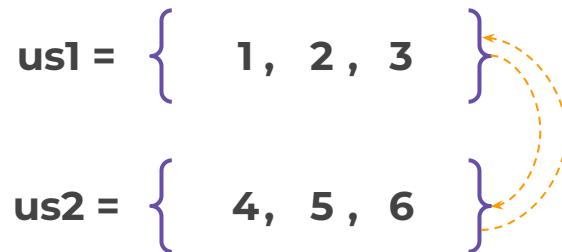
us1 = { 1, 2, 3 }

us2 = { 4, 5, 6 }

Swapping Two Unordered Sets

C++ 

```
unordered_set<int> us1 = {1, 2, 3};  
unordered_set<int> us2 = {4, 5, 6};  
us1.swap(us2);
```



Swapping Two Unordered Sets

C++ 

```
unordered_set<int> us1 = {1, 2, 3};  
unordered_set<int> us2 = {4, 5, 6};  
us1.swap(us2);
```

us1 = { 4, 5, 6 }

us2 = { 1, 2, 3 }

Quiz Time!



C++ 

```
#include <bits/stdc++.h>
using namespace std;

class UnorderedSetOperations {
public:
    unordered_set<int> us;

    void insert(int value) {
        us.insert(value);
    }
}
```

C++ 

```
void erase(int value) {
    us.erase(value);
}

void findElement(int value) {
    if(us.find(value) != us.end())
        cout << "Present\n";
    else
        cout << "Not Present\n";
}
};
```

Key Takeaways

✓ **Set**

Stores **unique** elements in **sorted order**, supports operations like **insert()**, **erase()**, **find()**, **size()**, and **swap()**.

✓ **Multiset**

Similar to **set**, but **allows duplicate** elements while maintaining **sorted** order. Provides **count()** for **duplicate** occurrences.



Key Takeaways

✓ ***Unordered Set***

Stores **unique** elements in an **arbitrary** order, offering generally **faster insertions** and **lookups** compared to **set**.

✓ ***Operations***

All **sets** support **insertion**, **deletion**, **iteration**, checking **existence**, **clearing**, **swapping**, and **size** retrieval.





THANK YOU





ALL THE BEST



Iterating Over an Unordered Set

C++ 

```
unordered_set<int> us = {5, 10, 15, 20, 25, 30};  
:  
:  
for (auto it = us.begin(); it != us.end(); it++)  
{  
    cout << *it << " ";  
}
```

us = {10, 5, 25, 30, 20, 35, 40}

Output 

10 5 25 30 20 35 40

Iterating Over a Multi set

C++ 

```
multiset<int> ms = {10, 5, 20, 15, 10};  
:  
:  
for (auto it = ms.begin(); it != ms.end(); it++) {  
    cout << *it << " ";  
}
```

$ms = \{ 5, 15, 20, 25, 30 \}$

Output 

5 15 20 25 30

Problem Statement

Your task is to perform the following operations on an unordered set:

- **Insertion**: Add elements to the unordered set.
- **Erase**: Remove elements from the unordered set.
- **Find**: Locate an element in the unordered set and print **“Present”** if it is present; otherwise, print **“Not Present”**.

Problem Statement

Input



Unordered Set 1: [1, 2, 2, 3, 4]

Operations:

insert 5

erase 2

find 3

Output



After Insertion: [1, 2, 3, 4, 5]

After Erasing 2: [1, 3, 4, 5]

Find 3: Present

Problem Statement

Insertion adds **5** to the **unordered set**, keeping only **unique values**

Result:

[1, 2, 3, 4, 5]

Erase removes **2** from the unordered set.

Result:

[1, 3, 4, 5]

Find locates **3** in the **unordered set** and confirms its presence.

Result:

Present